Technology as Seva: Implementation Blueprint

Swami Vivekananda Innovation Hackathon. Core philosophy: **Human ↔ Human. Contribution ↔ Learning.**

0. Final Positioning (read this first)

> **Technology is not the Seva. Technology is the enabler that helps human beings serve, learn, empower one another and continue the cycle of Seva.**

One-line pitch: A platform where nobody is a volunteer and nobody is a beneficiary. Everyone is a participant who contributes something and learns something, and the product is designed to make that exchange continue.

Three-sentence pitch for judges:

1. Most platforms ask "who can help whom?" We ask "what can each person contribute, and what can each person learn?"
2. Every feature is built to remove hierarchy (no giver/receiver roles, no rankings, no public hour counts), to build capability instead of dependency (every need is checked for self-reliance), and to turn one-off acts into continuing communities (Seva Circles and Seva Chains).
3. The AI never scores anyone. It holds up a mirror to private reflections so a young person can see how they are changing.

The 8-step loop (visible in navigation, onboarding and the demo): FEEL → DISCOVER → CONNECT → SERVE → LEARN → REFLECT → EMPOWER → CONTINUE

---

1. Product Vision

The platform itself is not the Seva. It enables Seva in the real world. Success is measured by what happens offline: a painting session held, a website handed over and understood by the people who run it, a farmer and a coder teaching each other.

Design stance:

- **Participant** is the only identity. Roles (creator, contributor, learner) are per-activity relationships, never properties of a person.
- **Need-first, not listing-first.** Communities state needs; the platform structures them into opportunities.
- **Private by default.** Reflection, journey and silent Seva are never public.
- **Explainable.** Every recommendation says why in a sentence.
- **Real-world first.** Every flow ends in an offline act and returns to reflection.

---

2. Philosophy Mapping

Vivekananda / Mission principle	Product requirement	Where it lives
Each soul is potentially divine	No label that ranks or pities a person; identical profile shape for all	Participant model, UI language rules
Service to man is worship of God	Respectful, dignity-first language; no poverty showcase	Content guidelines, moderation, consent
The giver receives more	Mandatory "What did I learn?" alongside "What did I contribute?"	Seva Moment, Contribution/Learning card
Work without attachment to results	Reflection on quality of action, not outcome; silent Seva; no public counters	Reflection prompts, Silent Seva, no leaderboards
Feel first, organize afterwards	Sankalp (intention) before matching; need posting starts as a free-text feeling	Sankalp, Create Need
Pride and superiority destroy service	Banned vocabulary (hero, top volunteer, beneficiary, saved); no badges	Design system lint rules
Excellence, efficiency, teamwork	Structured needs, clear roles per activity, circles with shared tasks	Need structuring, Circles

Vivekananda / Mission principle	Product requirement	Where it lives
Truthfulness and transparency	Organization verification, aggregate public impact, honest moderation log	Admin, org verification
Empowerment, avoid dependency	"Can this increase the community's capability?" check on every need and seva	Empowerment check field
Demand and supply (relief model)	Needs assessed first, then capabilities mobilized	Needs → matching
Long-term service, man-making	Circles, Chains, journey, continuity metrics	Circles, Chain, Journey
Mutual help, equitable opportunity	Skill Exchange with I can contribute / I want to learn	Skill Exchange

---

3. Feature Architecture

Every major feature in the format: **Feature → User problem → Principle → Product behavior → Technical implementation → MVP priority.**

#	Feature	User problem	Principle	Product behavior	Technical implementation	Priority
1	Contribution Profile	"I don't know what I have to offer, or what I could learn."	Potential divinity, equality	Two equal lists: I can contribute / I want to learn, plus availability and area	`profiles`, `user_skills` with `kind` = contribute or learn	**MVP**
2	Sankalp	"I do Seva but forget why."	Feel first; inner discipline	Pick one intention (patience, listening, humility…) or write your own; changeable; private	`sankalps` table; tags used as a soft input to recommendations	**MVP**

#	Feature	User problem	Principle	Product behavior	Technical implementation	Priority
3	Community Needs	"Organizations want volunteers; communities want specific things."	Demand and supply	Post a need in plain words; AI structures it (required contributions, sessions, potential learning, empowerment check)	`community_needs`, LLM structuring endpoint with human edit before publish	**MVP**
4	Seva Discovery	"Hard to find something real, nearby, fitting me."	Organize after feeling	Filters: location, category, skill, date, contribution type, learning opportunity	Postgres query + PostGIS-lite distance (or lat/lng + haversine)	**MVP**
5	Create Seva	"I have a skill and a free Saturday but no structure."	Excellence, efficiency	Form with purpose, required contributions, skills, safety info, learning potential	`seva_opportunities`, `seva_events`	**MVP**
6	Explainable Recommendations	"Matching tools feel like a black box."	Truthfulness	One human sentence per match; no percentages	Rule engine returns `reasons[]`	**MVP**
7	Technology as Seva	"Tech skills are used for freelancing, not service."	Capability → service	Tech needs have a mandatory handover-and-teach line; no bidding, no prices	Category flag `tech`, empowerment check required	**MVP**
8	Seva Circles	"After the event, the group dissolves."	Long-term service	Auto-offer circle after a Seva; discussion, announcements, resources, next Seva	`seva_circles`, `circle_members`, `messages` (WebSocket)	**MVP (lite)**
9	Seva Moment	"I did something but never processed it."	Giver receives more	Contribution + Learning + surprise + what the community taught me	`seva_moments`, `reflections`	**MVP**
10	AI Mirror	"I can't see how I'm changing."	Self-knowledge, not judgment	Patterns from private reflections; never a score	LLM over the user's own reflections, strict prompt, stored in `ai_insights`	**MVP (basic) / Phase 2 (deep)**
11	Seva Journey	"Apps show points; I want to see growth."	Man-making	Timeline: sankalp → seva → learnings → circles → recurring themes	Aggregation over own data; private	**MVP**

#	Feature	User problem	Principle	Product behavior	Technical implementation	Priority
12	Skill Exchange	"I want to learn farming, I can teach coding."	Equality	Reciprocal matches: A teaches what B wants, B teaches what A wants	Two-sided rule match on `user_skills`	**MVP**
13	Seva Chain	"Help ends with one act."	Mutual help	Optional link: "this learning led to that contribution"	`seva_chain_links` graph, simple visualization	**Phase 2 (demo mock in MVP)**
14	Silent Seva	"I don't want recognition."	Non-attachment	Log Seva with no visibility, no counts shown to others	`visibility = private` default, excluded from all aggregates shown to users	**MVP**
15	Safety & Consent	"Meeting strangers; vulnerable communities."	Dignity	Consent before contact, report, location rounding, no photos of individuals in need	`consents`, `reports`, location fuzzing	**MVP**
16	Verification	"Is this real?"	Truthfulness	Creator + participant confirmation, optional evidence, organization badge; no surveillance	Event completion state machine	**MVP (light)**
17	Admin	"Keep the platform safe and healthy."	Transparency	Moderate, review reports, aggregate impact	Role-gated admin routes	**MVP (minimal)**

---

4. MVP vs Phase 2 vs Future

MVP (hackathon demo, the primary loop): Join → contribution profile → Sankalp → discover a need → explainable recommendation → join/create Seva → circle chat → real-world act → Seva Moment → reflection → AI Mirror (basic) → Journey → continue (circle or next Seva).

Phase 2: Seva Chain visualization with real data, deeper AI Mirror (pattern analysis over time), multilingual UI and AI assistance, push notifications, organization onboarding, PWA offline for low-connectivity.

Future: Learned recommendations (still explainable), community-level insights for organizations, Mission-partner integrations, large-scale aggregate impact reporting, mentorship pathways.

Cut list if time runs out: Admin analytics charts, Seva Chain graph (keep mock), notifications beyond in-app, messaging attachments.

---

5. User Personas

No persona is a "beneficiary". Each contributes and learns.

1. **Aarav, 20, CS student.** Wants to use coding for something meaningful. Contributes: frontend. Learns: public speaking. Sankalp: humility. Risk: treating Seva as a portfolio item.
2. **Meera, 24, UI designer.** Contributes: design. Learns: Hindi calligraphy, teaching. Sankalp: listening.
3. **Savita, 52, runs a community school in a village-edge locality.** Posts a need for digital presence. Contributes: teaching experience, local knowledge, farming knowledge. Learns: basic computing.
4. **Ramesh, 60, retired farmer.** Contributes: farming. Learns: smartphone use. Part of a Seva Chain.
5. **Ishita, 19, first-year student, anxious about trying.** Needs a low-pressure first Seva: small, local, with a circle.
6. **Admin / Moderator, Priya.** Needs fast report triage and aggregate health numbers, never individual rankings.

Personas are for design only; the data model has no "persona" or "role of person".

---

6. User Journeys

Journey A: Technology Seva (primary demo)

1. Savita posts a need in plain words: "Our school needs a simple website."
2. The platform structures it: required contributions (frontend, design, content), 4 sessions, potential learning (community needs, communication), empowerment check: "Will someone at the school be able to update it?" Savita confirms, adding a handover session.

3. Aarav completes his profile, picks Sankalp "humility", and sees the need with the reason: *"Recommended because you can contribute web development and this community needs a website."*
4. Meera and a content contributor see similar reasons. All three join and consent to share contact within the circle.
5. A Seva Circle opens. They plan, meet, and build.
6. After the handover session, each records a Seva Moment.
7. Aarav's AI Mirror shows a pattern from his own words. His Journey updates.
8. The circle proposes a next Seva: a digital literacy session for parents.

Journey B: Skill Exchange. Participant A (teaches coding, wants farming) and Participant B (teaches farming, wants coding) are matched by a two-sided rule. They agree on two sessions at B's field and A's laptop. Both record what they contributed and learned. No one is a giver.

Journey C: Silent Seva. Ishita cleans a lane near her home with neighbors, logs it as private. No counter appears anywhere. It appears only in her own Journey, if she chooses.

---

7. Information Architecture

```
Public
  Landing, Philosophy (Why Seva), How It Works,
  Explore Seva, Explore Community Needs, Technology as Seva

Authenticated
  Onboarding -> Contribution Profile -> Sankalp
  Home (the loop: where am I in Feel->Continue?)
  Discover (Seva | Needs | People for exchange)
  Community Need Detail | Seva Detail
  Create Seva | Create Community Need
  Skill Exchange
  Participant Profile (limited public view)
  Seva Circle (+ Messages)
  Seva Moment -> Reflection -> AI Mirror
  Seva Journey
  Silent Seva
  Notifications
```

```
Admin
  Dashboard, User Moderation, Seva Moderation,
  Need Moderation, Reports, Analytics
```

Primary mobile navigation (5 tabs): Home, Discover, Circles, Journey, You. Create is a persistent action, not a tab.

Home screen shows the loop as a path with the user's current position (e.g. "You've served; reflect next"). No streaks, no counts.

---

8. Complete Page List (31 pages)

Public (6): Landing, Philosophy, How It Works, Explore Seva, Explore Community Needs, Technology as Seva.

Authenticated (19): Onboarding, Contribution Profile, Sankalp, Home Dashboard, Discover, Community Need Detail, Seva Detail, Create Seva, Create Community Need, Skill Exchange, Participant Profile, Seva Circle, Messages, Seva Moment, Reflection, AI Mirror, Seva Journey, Silent Seva, Notifications.

Admin (6): Admin Dashboard, User Moderation, Seva Moderation, Community Need Moderation, Reports, Analytics.

MVP page set (build these first, 14): Landing, Onboarding, Contribution Profile, Sankalp, Home, Discover, Community Need Detail, Seva Detail, Create Need, Create Seva, Circle (with chat), Seva Moment + Reflection, AI Mirror, Journey. Skill Exchange can be a Discover tab for the second demo. Admin is one page with report triage.

---

9. The Eight-Question Product Test (apply at every review)

1. Does it preserve dignity?
2. Does it create a giver/receiver hierarchy?
3. Does it encourage selfless action or ego?
4. Does it empower or create dependency?
5. Does it help people continue Seva?
6. Does it enable genuine human connection?
7. Does technology add meaningful value?

8. Would it still make sense without the words "Swami Vivekananda"?

Applied to rejected ideas (examples for judge Q&A):

- *Points and leaderboards:* fails tests 2 and 3. Removed.
- *"Hours served" counter on profiles:* fails test 3. Kept only privately in Journey, as optional.
- *Photo stories of people helped:* fails test 1. Replaced by the community's own words, with consent.
- *Freelance bidding for tech needs:* fails tests 2, 3, 8. Replaced by capability-to-need matching with handover.
- *"Match score 92%":* fails test 7 (no meaningful value, no explanation). Replaced by a human-readable reason.

---

10. Language and Design Rules (enforced in the design system)

Use	Never use
Participant	Volunteer, beneficiary, receiver, needy, helper
Community, Need, Opportunity	Cause, victim, case
Contribution, Learning	Donation, charity, rescue
"Joined", "took part"	"Saved", "helped out"

Visual direction: calm, minimal, warm neutrals with a single restrained accent (deep teal or indigo, not saffron). Large readable type, generous whitespace, photography of hands and activity, never of individuals in distress. Interaction carries the philosophy: the Seva Moment screen always shows Contribution and Learning side by side with equal visual weight.

Lint rule (cheap, strong demo point): a CI script greps the codebase and copy files for banned words and fails the build.

---

11. Frontend Architecture

Stack (final): Next.js 14 (App Router) + React 18 + TypeScript (strict) + Tailwind CSS + shadcn/ui (Radix primitives, accessible by default) + TanStack Query (server state) + Zustand (tiny UI state only) + React Hook Form + Zod (validation shared with API types) + Socket.IO client (circle chat) + next-intl (i18n-ready from day one) + PWA via `next-pwa` (installable, offline shell; Phase 2 for offline data).

Why this stack: Next.js gives SEO for public pages (Explore, Philosophy) and fast client navigation for authenticated pages; Tailwind + shadcn lets four students build a consistent design system quickly; Zod lets the frontend validate with the same schemas the backend enforces.

Folder structure:

```
/apps/web
  /app
    /(public)        landing, philosophy, how-it-works, explore, tech-as-seva
    /(auth)          login, signup, onboarding
    /(app)           home, discover, needs/[id], seva/[id], create/*,
                     exchange, people/[id], circles/[id], moment/[id],
                     mirror, journey, silent, notifications
    /(admin)         admin/*
  /components
    /ui              shadcn primitives
    /seva            SevaCard, ContributionLearningCard, ...
    /layout          AppShell, BottomNav, TopBar
  /lib
    api-client.ts    typed fetch wrapper (auth, errors, pagination)
    types.ts         generated from OpenAPI / shared package
    banned-words.ts  copy lint list
  /mocks             MSW handlers for every endpoint (build before backend)
  /styles            tokens.css (design tokens)
/packages/contracts  shared Zod schemas + TS types (frontend and backend
import)
```

Rendering strategy: public pages are static/ISR; authenticated pages are client components behind a session guard using TanStack Query; the circle chat uses a WebSocket.

Responsive and accessibility rules: mobile-first (375px baseline), bottom navigation on mobile and sidebar on desktop, WCAG AA contrast, visible focus rings, every control reachable by keyboard, `aria-live` on chat and notifications, no meaning conveyed by

color alone, minimum 44px touch targets, `prefers-reduced-motion` respected.

Standard states every data view must implement (reviewed in PRs):

- *Loading:* skeleton matching the final layout, never a lone spinner.
- *Empty:* a message that invites the next step in the loop (e.g. "No needs near you yet. Describe one you feel.").
- *Error:* plain language, retry button, no stack traces.
- *Offline:* banner, queued reflection drafts saved locally.

---

12. Frontend Component Architecture

Design tokens first (`tokens.css`): `--color-bg` , `--color-surface` , `--color-ink` , `--color-accent` , `--color-contribution` , `--color-learning` (the last two have equal visual weight, always used as a pair), spacing scale 4/8/12/16/24/32, radius 12/16, one display serif for headings and one sans for body.

Component	Purpose and rules
SevaCard	Title, place (rounded), date, contribution chips, *learning* chips, and the recommendation reason line. No participant counts as social proof.
CommunityNeedCard	Need in the community's own words, structured needs, empowerment line ("leaves the school able to update the site itself").
ParticipantCard	Name, "can contribute" and "wants to learn" chips side by side. Never shows hours or ratings.
SkillChip	Variants: `contribute` , `learn` . Same shape and weight.
ContributionCard / LearningCard	Paired component, rendered together in Seva Moment and Journey.
ContributionLearningComparison	Two-column view for exchange: "A offers / B offers" and "A learns / B learns".

Component	Purpose and rules
SankalpSelector	Chips for patience, listening, humility, courage, compassion, discipline, responsibility, selflessness, plus free text. Copy: "What do you want to practice?"
SevaJourneyTimeline	Vertical timeline: sankalp → seva → learning → circle → reflection, with recurring themes in a side rail.
ReflectionCard	Prompt + textarea + "private" lock indicator (always visible).
SevaCircleCard	Name, shared purpose, next gathering, last announcement.
SevaChainVisualization	Linked nodes: contribution → learning → contribution. SVG, d3 optional. Phase 2 with mock data in MVP.
LocationCard	Neighborhood-level location; exact address shown only after consent to join.
EventCard	Date, time, place, safety note, what to bring.
RecommendationExplanation	A sentence starting "Recommended because…"; expands to show which inputs were used.
AIMirrorInsight	Observation + quoted words from the user's own reflection + "Does this feel true?" response (this feedback tunes the prompt).
Notification	Calm, text-first, no badges for "likes" (none exist).
ConsentModal	Plain-language consent before contact or sharing; separate toggles; withdraw anytime.
ReportModal	Reason list + free text + "also block"; confirmation of next steps.

Storybook (or a `/design` route) documents each component with its loading, empty and error variants.

---

13. Backend Architecture

Stack: Node.js 20 + TypeScript, **NestJS** (modular, good for two backend students splitting modules) or Express + Zod if the team prefers lighter. PostgreSQL 15 with Prisma ORM. Redis (sessions/rate limits/socket adapter, optional for MVP). Socket.IO for real-time. An LLM provider behind an `AIService` interface (provider swappable). Object storage (S3-compatible) for optional evidence images.

Modular monolith (right size for a hackathon):

```
/apps/api/src
  auth/           signup, login, JWT, refresh, roles
  profiles/       profile, contribution profile, visibility
  skills/         catalog + user_skills (contribute | learn)
  sankalp/
  needs/          community needs + AI structuring
  seva/           opportunities, events, join/leave, completion
  exchange/       two-sided skill matching
  matching/       rule engine -> reasons[]
  circles/        circles, members, announcements
  messages/       REST history + WebSocket gateway
  moments/        seva moments + reflections
  journey/        read-model aggregation (own data only)
  ai/             AIService (structuring, summarize, mirror) + guardrails
  notifications/
  moderation/     reports, actions, audit log
  consents/
  admin/          aggregate analytics
  common/         guards, pipes, error filter, pagination, logging
```

Cross-cutting rules:

- Every request body validated by Zod schemas from `/packages/contracts`.
- Resource-level authorization checked in a service layer, not only route guards.
- All list endpoints cursor-paginated.
- Structured JSON logging with request IDs; no reflection text in logs.
- Reflections and AI insights encrypted at rest at the column level (pgcrypto or app-level AES) as a visible privacy feature.

---

14. Database Schema (PostgreSQL)

Design rules: no `beneficiary_id`, no `receiver_role`, no permanent "giver" flag. Relationship to an activity lives in `participants.stance` per activity. Money is deliberately absent from the model.

```
-- Identity
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email CITEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  system_role TEXT NOT NULL DEFAULT 'participant'
    CHECK (system_role IN ('participant','moderator','admin')),
  status TEXT NOT NULL DEFAULT 'active'
    CHECK (status IN ('active','suspended','deleted')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE profiles (
  user_id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  display_name TEXT NOT NULL,
  bio TEXT,
  area_label TEXT,              -- neighborhood/city level only
  lat NUMERIC(8,5), lng NUMERIC(8,5),  -- rounded, never exposed raw
  languages TEXT[] DEFAULT '{}',
  availability JSONB,           -- e.g. {"weekends":true,"evenings":true}
  visibility TEXT NOT NULL DEFAULT 'members'
    CHECK (visibility IN ('private','members','public')),
  onboarding_done BOOLEAN DEFAULT false
);

-- Skills: one catalog, two directions
CREATE TABLE skills (
  id SERIAL PRIMARY KEY,
  name TEXT UNIQUE NOT NULL,
  category TEXT NOT NULL        -- tech, art, teaching, farming, ...
);
CREATE TABLE user_skills (
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  skill_id INT REFERENCES skills(id),
  kind TEXT NOT NULL CHECK (kind IN ('contribute','learn')),
  level TEXT,                   -- optional: beginner/practiced/experienced
  PRIMARY KEY (user_id, skill_id, kind)
```

```
);
CREATE TABLE contribution_profiles (  -- other resources a person can offer
  user_id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  can_offer_time BOOLEAN DEFAULT true,
  can_offer_knowledge BOOLEAN DEFAULT true,
  can_offer_resources TEXT[],   -- e.g. {'laptop','paints','space'}
  notes TEXT
);

CREATE TABLE sankalps (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  intention TEXT NOT NULL,      -- 'listening', 'patience', or custom
  note TEXT,
  active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE organizations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  description TEXT,
  area_label TEXT,
  verified BOOLEAN DEFAULT false,
  verified_by UUID REFERENCES users(id),
  created_by UUID REFERENCES users(id)
);

-- Needs and Seva
CREATE TABLE community_needs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  posted_by UUID REFERENCES users(id),
  organization_id UUID REFERENCES organizations(id),
  raw_text TEXT NOT NULL,       -- the community's own words, preserved
  title TEXT NOT NULL,
  structured JSONB NOT NULL,    -- required contributions, sessions,
potential learning
  empowerment_note TEXT NOT NULL, -- how this builds capability (required)
  category TEXT, area_label TEXT, lat NUMERIC(8,5), lng NUMERIC(8,5),
  status TEXT DEFAULT 'open'
    CHECK (status IN
('draft','open','in_progress','fulfilled','closed','removed')),
  created_at TIMESTAMPTZ DEFAULT now()
```

```
);

CREATE TABLE seva_opportunities (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  created_by UUID REFERENCES users(id),
  need_id UUID REFERENCES community_needs(id),   -- nullable
  organization_id UUID REFERENCES organizations(id),
  title TEXT NOT NULL, purpose TEXT NOT NULL, description TEXT,
  category TEXT, is_tech BOOLEAN DEFAULT false,
  required_contributions JSONB,  -- [{skill_id, count, description}]
  potential_learning TEXT[],
  resources_needed TEXT[],
  capacity INT,
  safety_info TEXT,
  empowerment_note TEXT NOT NULL,
  status TEXT DEFAULT 'open'
    CHECK (status IN
('draft','open','full','completed','cancelled','removed')),
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE seva_events (            -- one opportunity may have many
sessions
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  opportunity_id UUID REFERENCES seva_opportunities(id) ON DELETE CASCADE,
  starts_at TIMESTAMPTZ NOT NULL, ends_at TIMESTAMPTZ,
  place_label TEXT, address_private TEXT,  -- revealed after consent
  lat NUMERIC(8,5), lng NUMERIC(8,5),
  mode TEXT DEFAULT 'in_person' CHECK (mode IN
('in_person','online','hybrid')),
  completed_at TIMESTAMPTZ
);

CREATE TABLE participants (           -- relationship per activity, not per
person
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  opportunity_id UUID REFERENCES seva_opportunities(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  stance TEXT NOT NULL DEFAULT 'both'
    CHECK (stance IN ('contributing','learning','both')),
  contributing_as TEXT,               -- free text: "frontend", "hosting"
  status TEXT DEFAULT 'joined'
    CHECK (status IN ('joined','left','completed')),
```

```
  contact_consent BOOLEAN DEFAULT false,
  joined_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE (opportunity_id, user_id)
);

CREATE TABLE resources (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  opportunity_id UUID REFERENCES seva_opportunities(id),
  offered_by UUID REFERENCES users(id),
  label TEXT NOT NULL, kind TEXT, note TEXT
);

-- Community
CREATE TABLE seva_circles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL, purpose TEXT,
  origin_opportunity_id UUID REFERENCES seva_opportunities(id),
  created_by UUID REFERENCES users(id),
  status TEXT DEFAULT 'active' CHECK (status IN
('active','paused','closed')),
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE circle_members (
  circle_id UUID REFERENCES seva_circles(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  joined_at TIMESTAMPTZ DEFAULT now(),
  muted BOOLEAN DEFAULT false,
  PRIMARY KEY (circle_id, user_id)
);
CREATE TABLE connections (            -- 1:1, consent-based
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  requester_id UUID REFERENCES users(id),
  addressee_id UUID REFERENCES users(id),
  status TEXT DEFAULT 'pending'
    CHECK (status IN ('pending','accepted','declined','blocked')),
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  circle_id UUID REFERENCES seva_circles(id) ON DELETE CASCADE, -- or
  connection_id UUID REFERENCES connections(id) ON DELETE CASCADE,
  sender_id UUID REFERENCES users(id),
  kind TEXT DEFAULT 'message' CHECK (kind IN
```

```
('message','announcement','resource')),
  body TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  CHECK ((circle_id IS NOT NULL) <> (connection_id IS NOT NULL))
);

-- Reflection and growth (all private by default)
CREATE TABLE seva_moments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  opportunity_id UUID REFERENCES seva_opportunities(id),  -- nullable (silent
seva)
  contributed TEXT NOT NULL,
  learned TEXT NOT NULL,
  taught_by_others TEXT,
  visibility TEXT NOT NULL DEFAULT 'private'
    CHECK (visibility IN ('private','circle')),
  is_silent BOOLEAN DEFAULT false,
  occurred_on DATE NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE reflections (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  moment_id UUID REFERENCES seva_moments(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  surprise TEXT, expectation_match TEXT, do_differently TEXT,
  sankalp_id UUID REFERENCES sankalps(id),
  body_encrypted BYTEA,         -- encrypted at rest
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE ai_insights (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  kind TEXT CHECK (kind IN ('summary','mirror','need_structuring')),
  content TEXT NOT NULL,
  source_ids UUID[],            -- which reflections it drew on
  user_feedback TEXT CHECK (user_feedback IN ('feels_true','not_quite')),
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE seva_chain_links (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  from_moment_id UUID REFERENCES seva_moments(id),
  to_moment_id UUID REFERENCES seva_moments(id),
```

```
  relation TEXT DEFAULT 'learning_led_to_contribution',
  confirmed_by_both BOOLEAN DEFAULT false
);

-- Safety
CREATE TABLE consents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  scope TEXT NOT NULL,          --
'share_contact_circle','profile_public',...
  granted BOOLEAN NOT NULL,
  context_id UUID,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE reports (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reporter_id UUID REFERENCES users(id),
  target_type TEXT CHECK (target_type IN
('user','seva','need','message','circle')),
  target_id UUID NOT NULL,
  reason TEXT NOT NULL, details TEXT,
  status TEXT DEFAULT 'open' CHECK (status IN
('open','reviewing','actioned','dismissed')),
  handled_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  type TEXT NOT NULL, payload JSONB, read_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON seva_opportunities (status, category);
CREATE INDEX ON community_needs (status, category);
CREATE INDEX ON user_skills (skill_id, kind);
CREATE INDEX ON messages (circle_id, created_at DESC);
```

Privacy-by-schema notes: `visibility` defaults are the most private option; `address_private` is only returned to participants with consent; `is_silent` moments are excluded from every aggregate query; admin analytics read from views that never include user IDs.

---

15. API Specification (REST, JSON, `/api/v1`)

Conventions:

- Auth: `Authorization: Bearer <access JWT>`. "Auth" column: **P** = public, **U** = signed-in participant, **M** = moderator, **A** = admin. "Own" means resource-level check that the caller owns or belongs to it.
- Pagination: `?limit=20&cursor=<opaque>` returns `{ data: [...], nextCursor: string | null }`.
- Errors always use the format in section 17.
- Dates are ISO 8601 UTC. IDs are UUIDs.

Auth and profile

Method	Route	Request body	Response	Auth	Purpose
POST	`/auth/signup`	`{email, password, displayName}`	`{user, accessToken}` + refresh cookie	P	Create account
POST	`/auth/login`	`{email, password}`	`{user, accessToken}` + refresh cookie	P	Sign in
POST	`/auth/refresh`	cookie	`{accessToken}`	P	Rotate token
POST	`/auth/logout`	none	`204`	U	Revoke refresh token
GET	`/profile`	none	`Profile` (own, full)	U	Load own profile
PATCH	`/profile`	partial `Profile`	`Profile`	U (Own)	Update profile, visibility
GET	`/participants/:id`	none	`PublicParticipant` (respects visibility)	U	View another participant
GET	`/skills`	`?q=&category=`	`Skill[]`	U	Skill catalog
POST	`/skills`	`{name, category}`	`Skill`	U	Suggest new skill (goes to moderation)
PUT	`/profile/skills`	`{contribute: SkillRef[], learn: SkillRef[]}`	`{contribute, learn}`	U (Own)	Replace contribution/learning lists
PUT	`/profile/contribution`	`ContributionProfile`	same	U (Own)	Time, knowledge, resources offered
POST	`/sankalp`	`{intention, note?}`	`Sankalp`	U	Set active intention (deactivates previous)
GET	`/sankalp`	none	`Sankalp[]`	U (Own)	History

Needs and Seva

Method	Route	Request body	Response	Auth	Purpose
GET	`/needs`	`?category=&near=lat,lng&radiusKm=&status=&cursor=`	`Page<CommunityNeed>`	P (public subset) / U	Browse needs
POST	`/needs/structure`	`{rawText}`	`StructuredNeedDraft`	U	AI turns plain words into structure (not saved)
POST	`/needs`	`CreateNeedInput` (includes `empowermentNote`)	`CommunityNeed`	U	Publish need (goes to light review if poster is new)
GET	`/needs/:id`	none	`CommunityNeedDetail`	P/U	Detail, linked Seva, contributions already offered
PATCH	`/needs/:id`	partial	`CommunityNeed`	U (Own)	Edit / close / mark fulfilled
POST	`/needs/:id/offer`	`{contribution, message?}`	`Participant`	U	Offer a contribution to a need (creates or joins its Seva)
GET	`/seva`	`?category=&skill=&date=&near=&radiusKm=&contribution=&learning=&cursor=`	`Page<SevaOpportunity>`	P/U	Discovery with filters
POST	`/seva`	`CreateSevaInput`	`SevaOpportunity`	U	Create Seva
GET	`/seva/:id`	none	`SevaDetail`	P/U	Detail (exact address only if consented participant)
PATCH	`/seva/:id`	partial	`SevaOpportunity`	U (Own creator)	Edit, cancel
POST	`/seva/:id/join`	`{contributingAs?, stance, shareContact: boolean}`	`Participant`	U	Join; records consent
POST	`/seva/:id/leave`	none	`204`	U (Own)	Leave without penalty
POST	`/seva/:id/complete`	`{eventId, note?}`	`SevaOpportunity`	U (creator)	Mark event completed
POST	`/seva/:id/confirm`	`{eventId, attended: boolean}`	`204`	U (participant)	Participant confirmation
GET	`/exchange/matches`	none	`ExchangeMatch[]`	U	Two-sided skill matches
POST	`/exchange/propose`	`{withUserId, offer: SkillRef, want: SkillRef, message}`	`Connection`	U	Propose an exchange
GET	`/recommendations`	`?limit=`	`Recommendation[]` (with `reasons[]`)	U	Explainable recommendations

Circles and messages

Method	Route	Request body	Response	Auth	Purpose
GET	`/circles`	none	`Page<CircleSummary>`	U	My circles
POST	`/circles`	`{name, purpose, originOpportunityId?, memberIds?}`	`Circle`	U	Create a circle (members must consent/accept)

Method	Route	Request body	Response	Auth	Purpose
GET	`/circles/:id`	none	`CircleDetail`	U (member)	Circle info and members
POST	`/circles/:id/join`	none	`204`	U	Join (invite or open circle)
POST	`/circles/:id/leave`	none	`204`	U (member)	Leave
GET	`/circles/:id/messages`	`?cursor=`	`Page<Message>`	U (member)	History
POST	`/circles/:id/messages`	`{body, kind?}`	`Message`	U (member)	Send (also pushed via socket)
POST	`/connections`	`{addresseeId, message}`	`Connection`	U	Request 1:1 contact (consent-based)
PATCH	`/connections/:id`	`{status}`	`Connection`	U (addressee)	Accept, decline, block

Reflection and growth

Method	Route	Request body	Response	Auth	Purpose
POST	`/moments`	`{opportunityId?, contributed, learned, takenFromOthers?, occurredOn, isSilent?}`	`Moment`	U	Record Seva Moment
POST	`/reflections`	`{momentId, surprise?, expectationMatch?, doDifferently?, sankalpId?}`	`Reflection`	U	Reflection; may trigger summary
POST	`/reflections/:id/summarize`	none	`{summary}`	U (Own)	AI summary of this reflection
GET	`/mirror`	`?window=30d`	`MirrorInsight[]`	U (Own)	AI Mirror patterns
POST	`/mirror/:id/feedback`	`{feelsTrue: boolean}`	`204`	U (Own)	Feedback on a mirror insight
GET	`/journey`	none	`JourneyTimeline`	U (Own)	Personal journey
POST	`/chain/links`	`{fromMomentId, toMomentId}`	`ChainLink`	U	Propose a chain link (needs both confirmations)
GET	`/chain`	none	`ChainGraph`	U (own nodes)	Chain visualization data
GET	`/notifications`	`?cursor=`	`Page<Notification>`	U	Inbox
PATCH	`/notifications/:id/read`	none	`204`	U (Own)	Mark read

Method	Route	Request body	Response	Auth	Purpose
POST	`/consents`	`{scope, granted, contextId?}`	`Consent`	U	Record or withdraw consent
POST	`/reports`	`{targetType, targetId, reason, details?}`	`Report`	U	Report content or person

Admin

Method	Route	Response	Auth	Purpose
GET	`/admin/reports`	`Page<Report>`	M	Triage queue
PATCH	`/admin/reports/:id`	`Report`	M	Action or dismiss with note
PATCH	`/admin/users/:id/status`	`User`	A	Suspend/restore (audit logged)
PATCH	`/admin/seva/:id/status`	`SevaOpportunity`	M	Approve, reject, remove
PATCH	`/admin/needs/:id/status`	`CommunityNeed`	M	Approve, reject, remove
POST	`/admin/organizations/:id/verify`	`Organization`	A	Verify organization
GET	`/admin/analytics/health`	`HealthMetrics` (aggregate only)	A	Platform health

---

16. Authentication and Authorization

Authentication flow:

1. Signup/login returns a short-lived **access JWT** (15 min) in the response body (kept in memory on the client) and a **refresh token** in an `httpOnly`, `Secure`, `SameSite=Lax` cookie (7 days, rotated on use, stored hashed server-side).
2. The API client attaches the access token; on `401 TOKEN_EXPIRED` it calls `/auth/refresh` once, then retries, otherwise routes to login.
3. Passwords: argon2id. Rate limit login (5/min per IP+email). Email verification before posting needs or creating Seva.
4. Optional later: Google sign-in, phone OTP for communities with low email use.

Authorization model (three layers):

- **System role:** participant, moderator, admin (guards on routes).
- **Resource relationship:** creator of a Seva, member of a circle, owner of a reflection (checked in services).
- **Visibility and consent:** profile `visibility`, `contact_consent`, `address_private` reveal rules.

Resource	Who can read	Who can write
Reflection / Mirror / Journey / Silent Seva	Owner only (admins cannot read content)	Owner
Seva Moment (circle visibility)	Owner + circle members if owner chose	Owner
Circle messages	Circle members	Members (author can delete own)
Seva exact address	Joined participants who granted consent	Creator
Public profile	Per `visibility`	Owner
Reports	Moderators/admins	Reporter creates; moderators update

Rule: there is no endpoint that returns a ranked list of people. The backend never computes a person-level aggregate for others.

---

17. API Contract Between Teams

Shared package `/packages/contracts` holds Zod schemas and inferred TypeScript types. Frontend validates forms with them; backend validates requests with them. OpenAPI JSON is generated from the same schemas (`zod-to-openapi`) and published as the single source of truth.

Core types (excerpt):

```
export type SkillRef = { skillId: number; level?: 'beginner' | 'practiced' |
'experienced' };
export type Stance = 'contributing' | 'learning' | 'both';

export interface Profile {
  userId: string; displayName: string; bio?: string; areaLabel?: string;
  languages: string[]; availability?: Record<string, boolean>;
  visibility: 'private' | 'members' | 'public';
  contribute: SkillWithName[]; learn: SkillWithName[];
  contribution: { time: boolean; knowledge: boolean; resources: string[] };
  activeSankalp?: Sankalp; onboardingDone: boolean;
}

export interface StructuredNeed {
  title: string;
  requiredContributions: { skillId?: number; label: string; count: number }
[];
  sessions: number; durationNote?: string;
  potentialLearning: string[];
  empowermentSuggestion: string; // AI-drafted; the poster must confirm or
edit
}

export interface Recommendation {
  kind: 'seva' | 'need' | 'exchange';
  targetId: string;
  reasons: { code: string; text: string }[]; // human-readable, never a score
}

export interface Moment {
  id: string; opportunityId?: string; contributed: string; learned: string;
  takenFromOthers?: string; occurredOn: string; isSilent: boolean;
}

export interface MirrorInsight {
  id: string; text: string; evidence: { momentId: string; excerpt: string }
[];
  createdAt: string;
}
```

Error format (every non-2xx response):

```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "Please check the highlighted fields.",
    "fields": { "empowermentNote": "Tell us how this will build the
community's own capability." },
    "requestId": "a1b2c3"
  }
}
```

Codes: `UNAUTHENTICATED`, `TOKEN_EXPIRED`, `FORBIDDEN`, `NOT_FOUND`, `VALIDATION_FAILED`, `CONFLICT`, `RATE_LIMITED`, `CONSENT_REQUIRED`, `AI_UNAVAILABLE`, `INTERNAL`. `message` is user-safe plain language.

Frontend rules for the contract:

- The frontend builds against **MSW mocks** generated from the same OpenAPI file from day one; swapping to the real API is a base-URL change.
- Backend publishes contract changes via PR to `/packages/contracts`, reviewed by both sides. Additive changes only after Day 2.
- `AI_UNAVAILABLE` must degrade gracefully: forms stay usable without AI structuring or summaries.
- Real-time events (Socket.IO, namespace `/circles`): client emits `circle:join {circleId}`, `message:send {circleId, body}`; server emits `message:new Message`, `circle:announcement Message`, `notification:new Notification`. Auth by access token in the handshake.

---

18. Matching Engine (explainable, rule-based for MVP)

Principle: output is a list of reasons in human language, never a number shown to users. An internal sort order may exist, but it is never displayed and never compared across people.

Inputs: my contribute skills, my learn skills, my sankalp, my availability and area, the Seva's/need's required contributions, potential learning, schedule, location, and my earlier Seva (continuity).

Rules (each rule yields a reason and a small internal weight):

Rule	Condition	Reason text template
R1 Skill fit	My contribute skill appears in `required_contributions`	"Recommended because you can contribute **{skill}** and this community needs **{need}**."
R2 Learning fit	My learn skill appears in `potential_learning`	"You said you want to learn **{skill]**, and this Seva includes it."
R3 Sankalp fit	Seva category's `interaction_tags` include my sankalp tag	"You are practicing **{intention}**; this involves a lot of {listening/patience...}."
R4 Nearby	Within radius of my area	"This is in your neighborhood ({areaLabel})."
R5 Availability	Event time overlaps my declared availability	"It fits your {weekend} availability."
R6 Continuity	I was in the origin circle of the parent need/Seva	"This continues what your circle began with {circle}."
R7 Exchange	Two-sided: A.contribute ∩ B.learn and B.contribute ∩ A.learn	"{Name} can teach farming and wants to learn coding, which you can teach."

Algorithm:

1. Candidate set: open items, not mine, not full, not in my past declines.
2. Evaluate rules; keep candidates with at least one **skill-or-learning** reason (R1, R2 or R7). Proximity alone never qualifies.
3. Sort by number and type of reasons (internal); diversify so the list includes at least one **learning-oriented** item and one that continues prior Seva.
4. Return top N with `reasons[]`.

Fairness guards: no individual is ranked against others in any view. Needs that nobody matches are surfaced to admins as a *community-capability gap* signal, not as a failure of the poster. Cold start: if a user has no skills yet, show newest nearby needs with the reason "Newly posted near you."

Later: embeddings over skills and need text to find non-obvious matches, still emitting reasons, with a human-checkable explanation.

---

19. AI Architecture

Hard rules (enforced in system prompts and code):

- No scores, ratings, rankings, rewards or moral judgments of any person.
- Mirror reads only the user's own reflections; it quotes their words and offers observations as questions, not verdicts.
- No user text is used for model training; provider set to no-retention; reflection text is not logged.
- AI is optional; every AI feature has a manual fallback.

Phase	Capability	Design
MVP	Need structuring	Prompt: given raw text, output JSON matching `StructuredNeed` (+ draft empowerment suggestion). Poster edits before publishing.
MVP	Event description assist	Rewrite rough notes into a clear description; the creator accepts or edits.
MVP	Reflection summary	2-3 neutral sentences of what the person wrote, no evaluation.
MVP (basic)	AI Mirror	After 2+ reflections, a single pattern observation citing excerpts.
Phase 2	Mirror over time	Themes across 30/90 days, e.g. shift from outcome-talk to process-talk.
Phase 2	Multilingual	Translate needs and messages; keep original text beside translation.
Phase 2	Need classification	Category and skill tags for discovery.
Future	Community insights	Aggregate, anonymized capability gaps for organizations.

Mirror prompt skeleton:

```
You are a mirror, not a judge. You will receive a person's private
reflections.
Task: notice patterns in what THEY wrote (themes repeated, changes in tone,
things they learned from the people they set out to serve).
Rules: never rate, score, compare to others, praise or criticize character;
never say they are good, humble, kind etc. Use "you mentioned..." and quote
short excerpts. Offer at most 2 observations, each as something they can
recognize or disagree with. End with one open question. Output JSON:
{"observations":[{"text":"...","evidence":
[{"reflectionId":"...","excerpt":"..."}]}],"question":"..."}
```

Code-level guardrail: after generation, a validator rejects outputs containing numbers-as-scores, superlatives about the person, or words from a banned list (`best`, `top`, `score`, `rank`, `humble person`…). On failure it retries once, then falls back to the summary-only response.

Demo example: *"You began by describing your technical contribution. In your latest reflection, most of what you wrote is about what the school's team taught you about their daily work. Does that match how it felt?"*

---

20. Real-Time and Notification Architecture

Real-time (Socket.IO): one namespace `/circles`, room per circle. Messages persist via REST/service first, then broadcast. Redis adapter when running more than one API instance. Typing indicators and read receipts are **not** built (less pressure, less surveillance). Fallback: polling every 10 s if sockets fail.

Notifications: an `events` bus inside the monolith emits domain events (`seva.joined`, `circle.message`, `need.matched`, `moment.reminder`); a notification service maps events to in-app notifications, with email for key events (Phase 2: web push).

Notification principles: no streak reminders, no guilt, no counts. A gentle one after a Seva: "Whenever you're ready, you can record what you learned." Users can mute per circle and set quiet hours. Digest option for weekly activity.

---

21. Privacy and Security

- **Consent before contact:** contact details and exact address are never revealed until the person grants `contact_consent` for that specific context; withdrawable.

- **Location protection:** store lat/lng rounded to ~500 m; show neighborhood label only; exact address only after joining with consent.
- **Private by default:** reflections, mirror insights, journey and silent Seva are private; admins cannot read their content.
- **Dignity in content:** no photos of identifiable people in need; storytelling in the community's own words with consent; moderation checklist rejects "poverty showcase" content.
- **Minors:** under-18 accounts (hackathon audience includes students) get restricted messaging (circle only, no 1:1), no exact address, and need guardian-visible event details; Seva with minors requires an organization or an adult co-facilitator.
- **Technical:** HTTPS everywhere, argon2id, JWT rotation, CSRF-safe cookie settings, input validation with Zod, parameterized queries (Prisma), rate limiting, helmet headers, column-level encryption for reflections, least-privilege DB role, secrets in env vars, dependency scanning in CI, audit log for admin actions.
- **Data rights:** export my data (JSON) and delete my account; deletion anonymizes circle messages.

---

22. Verification of Real-World Seva (light-touch)

Goal: reasonable trust without bureaucracy or surveillance.

1. **Creator marks the event completed.**
2. **Participants confirm** "I took part" with one tap (needed for the circle to unlock Seva Moment prompts and for chain links).
3. **Organization verification** (admin-approved) adds a "verified organization" mark to needs from NGOs, schools, Mission centers.
4. **Optional evidence:** a photo of the activity or materials (not of beneficiaries) may be attached by the creator; never required.
5. **Community feedback:** the person or community who posted the need can add a short note: "What did this change for us?" Visible only to the circle.
6. **Anomaly review (admin side):** repeated cancellations or reports trigger moderator review, not automated punishment.

Nothing here produces points, public counts or ranks.

---

23. Moderation and Admin Dashboard

Moderation workflow: report → triage queue (severity: safety, dignity, spam, other) → moderator reviews content (not private reflections) → action (dismiss / warn / remove / suspend) → reporter notified of outcome → audit entry. New posters' first need and first Seva go to light pre-review; established ones are post-moderated.

Dignity checklist for needs and Seva: uses community's own words? No degrading description or photos? Has an empowerment note? Avoids pity or savior language? Has a safety note where relevant?

Admin dashboard (aggregate, never personal ranking):

- Open reports and median resolution time
- Needs posted, matched, in progress, fulfilled
- Seva that continued beyond the first event
- Active circles and circles that created a second activity
- Skills exchanged (count of reciprocal exchanges)
- Capability gaps: skills most requested vs available by area
- Reflection participation rate (percent of completed Seva with a moment)
- Safety signals (cancellations, repeat reports)

Admin does not see: reflection content, mirror insights, silent Seva, or individual hour counts.

---

24. Team Division

Person	Owns	Key deliverables
FE1	Landing + design system	Tokens, shadcn theme, AppShell, Landing, Philosophy, How It Works, Technology as Seva, banned-word lint, Storybook
FE2	Discover, Needs, Create	Discover (Seva/Needs), Need Detail, Seva Detail, Create Need (with AI structuring step), Create Seva, recommendation reason UI
FE3	People and community	Onboarding, Contribution Profile, Sankalp, Participant Profile, Skill Exchange, Circle + chat, Messages, Notifications

Person	Owns	Key deliverables
FE4	Reflection and growth	Seva Moment, Reflection, AI Mirror, Journey, Silent Seva, Seva Chain visual, Admin pages
BE5	Foundation	Repo setup, auth, DB schema/migrations, profiles, skills, sankalp, consents, seed data, CI
BE6	Core domain	Needs, Seva, matching, exchange, circles, messaging (socket), moments/reflections, journey, AI service, notifications, moderation/admin

If fewer people: 4 people: FE1+FE2 / FE3+FE4 / BE5 / BE6. 3 people: one full-stack lead on contracts + auth, one on Needs/Seva/matching, one on reflection/AI/journey. Always keep one person owning the design system and demo polish.

Rituals: 15-min standup morning and evening; contract changes announced in one channel; one person is demo director from Day 2.

---

25. Development Order

Assumes a 2-3 day hackathon plus prep. Scale hours as needed.

Phase A: Foundations (parallel, first 20%)

- BE5: monorepo, Postgres, Prisma schema, auth, seed data. BE6: OpenAPI contract draft.
- FE1: design tokens, AppShell, landing. FE2-4: MSW mocks from the contract and first screens.
- *Exit check:* sign up / log in works on the real API; all screens run on mocks.

Phase B: Core loop (middle 40%)

1. Contribution profile + Sankalp (FE3/BE5)
2. Need creation with AI structuring, Need Detail, Discover (FE2/BE6)
3. Matching and explainable reasons (BE6/FE2)
4. Join Seva + consent + circle creation + chat (FE3/BE6)

- *Exit check:* the Technology Seva demo works from need to circle chat on the real backend.

Phase C: Reflection and continuity (next 25%) 5. Seva Moment + Reflection (FE4/BE6) 6. Journey aggregation + AI Mirror basic (FE4/BE6) 7. Skill Exchange match + propose (FE3/BE6) 8. Continue prompt: "Propose next Seva to your circle" (FE3/FE2)

Phase D: Polish (final 15%)

- Seed realistic data for both demos, empty/error states, accessibility pass, admin report triage, Seva Chain visual, deploy, rehearse. Feature freeze at the start of this phase; only bug fixes after.

---

26. Git and GitHub Strategy

- **Branches:** `main` (always deployable, protected), `develop` (integration), feature branches `feat/<area>-<short-name>` (e.g. `feat/needs-structuring`), `fix/...`, `contract/...` for API contract changes.
- **Rules:** PR into `develop`, at least one review from the other team for anything touching `/packages/contracts`; squash merge; merge `develop` to `main` at each phase exit and before demos.
- **Commits:** conventional commits (`feat:`, `fix:`, `docs:`, `chore:`).
- **CI (GitHub Actions) on every PR:** install, typecheck, lint, banned-word check, unit tests, build, contract validation (OpenAPI diff, fail on breaking change after Day 2).
- **Issues and board:** one GitHub Project board with columns Backlog, In progress, Review, Done; each issue links to a feature row in section 3.
- **Code ownership:** `CODEOWNERS` maps folders to the person above.

---

27. Environment Setup

Monorepo (pnpm workspaces): `apps/web`, `apps/api`, `packages/contracts`.

```
# prerequisites: Node 20, pnpm, Docker
git clone <repo> && cd technology-as-seva
pnpm install
cp .env.example .env            # fill values
docker compose up -d db redis   # Postgres 15 + Redis
pnpm --filter api prisma migrate dev
pnpm --filter api prisma db seed
pnpm dev                        # web :3000, api :4000
```

`.env.example` (keys only):

```
DATABASE_URL=postgresql://seva:seva@localhost:5432/seva
REDIS_URL=redis://localhost:6379
JWT_ACCESS_SECRET=
JWT_REFRESH_SECRET=
ENCRYPTION_KEY=              # 32 bytes, reflections at rest
LLM_PROVIDER=anthropic
LLM_API_KEY=
APP_BASE_URL=http://localhost:3000
NEXT_PUBLIC_API_URL=http://localhost:4000/api/v1
NEXT_PUBLIC_USE_MOCKS=false  # true: frontend runs on MSW only
```

Seed data (critical for the demo): 30 skills across tech, art, farming, teaching; 8 participants with believable profiles; the school-website need; the painting session; the coding-farming pair; one completed Seva with moments so Journey and Mirror are not empty.

---

28. Deployment Strategy

- **Frontend:** Vercel (preview deployment per PR).
- **API + sockets:** Render or Railway (single container, WebSocket-capable), health endpoint `/health`.
- **Database:** managed Postgres (Neon, Supabase or Railway) with daily backups; migrations run on deploy.
- **Redis:** Upstash, optional for MVP (needed only for multi-instance sockets).
- **Secrets:** platform env vars, never in repo. Separate `staging` and `production` environments; the demo runs on a stable branch/tag.
- **Monitoring:** request-ID logging, Sentry (frontend and API), uptime ping.
- **Demo safety net:** a recorded walkthrough video, a local docker-compose bundle on a laptop, and seeded accounts with a printed credentials card, in case venue Wi-Fi fails.

---

29. Testing Strategy

Layer	What	Tool	Must-have tests
Unit	Matching rules, AI output validator, authorization policies	Vitest/Jest	Each rule R1-R7 produces the right reason; guardrail rejects score-like text; reflection owner-only
API	Endpoint contracts, validation, errors	Supertest	Auth flow, join with consent, address reveal only after consent, pagination
Contract	Frontend mocks match backend	OpenAPI diff in CI	No breaking change
Component	UI states	Testing Library + Storybook	Loading, empty, error for each data view
E2E	Primary demo loop	Playwright	Sign up → profile → sankalp → need → recommendation → join → chat → moment → mirror → journey
Accessibility	Keyboard + contrast	axe in CI	No critical violations on MVP pages
Philosophy	Banned words, no ranking endpoints	Lint script + review checklist	CI fails on "volunteer"/"beneficiary" in UI copy; no endpoint returns people sorted by activity
Privacy	Leakage	Manual + automated	Another user cannot read a reflection; admin cannot read reflection text

---

30. Success Metrics (and why each is Vivekananda-aligned)

Metric	Why it fits
% of participants who join a second Seva	Long-term service, not one-off charity

Metric	Why it fits
Number of circles that organize a second activity	Seva as a continuing community
Contribution-learning exchanges completed	Mutual help, no giver/receiver
Participants who moved from learning to contributing (and vice versa)	Potential divinity in everyone; dissolves hierarchy
Needs matched and fulfilled	"Feel first, organize afterwards" actually completing
Needs where the community reports increased capability	Empowerment over dependency
Participants who create their own Seva or need	Man-making: from participant to initiator
Reflection participation (completion of moments)	The giver receives more; inner growth
Time-to-first-Seva for new participants	Removing friction to feeling-into-action

Not measured or displayed: hours, points, likes, followers, per-person rankings.

---

31. Hackathon Demo Flow (7 minutes)

Time	Beat	What the judges see
0:00	**The problem**	Show a typical volunteer platform: "hours," leaderboard, "beneficiary." Then say the principle: *they alone live who live for others*, and ask what a product would look like that took that seriously.
0:45	**Feel**	Savita posts a need in plain words. AI structures it; she confirms the empowerment line ("someone at our school will maintain it").

Time	Beat	What the judges see
1:45	**Discover and connect**	Aarav (developer, sankalp: humility) opens Home. The reason line appears: *"Recommended because you can contribute web development and this community needs a website."* Meera and a content contributor see the same. They join and grant consent.
2:45	**Circle**	A Seva Circle opens; a short chat; plan for 4 sessions including a handover session.
3:30	**Serve (real world)**	Photo of the actual team (rehearsed, or a pre-recorded 15-second clip of a real session if the team ran one).
4:00	**Learn and reflect**	Aarav records Contribution and Learning side by side. The AI Mirror shows the pattern from his own words. Journey updates; no score, no badge.
5:00	**Continue**	The circle proposes the next Seva: digital literacy for parents. A Seva Chain tile appears.
5:30	**Second scenario**	Skill Exchange: coder wants farming, farmer wants coding. "No giver. No receiver. Only participants."
6:15	**Close**	Show the CI lint failing on the word "beneficiary", the privacy rule (admins cannot read reflections), and the line: *Technology is not the Seva. Technology is the enabler.*

Tips: run one real mini Seva before the demo (even two teammates teaching a neighbor) and use real words from it; keep a pre-seeded account that already has a Journey; rehearse the offline fallback.

---

32. Competitive Differentiation

Category	What they do	Where they fall short for this brief	What we do differently
Volunteer platforms	Match volunteers to NGO tasks, track hours	Fixed volunteer/beneficiary roles, hour counts and rankings, one-off tasks	One identity (participant), no counts, Sankalp and reflection, continuation via circles
Donation platforms	Money to causes	Giver/receiver by design; stories can exploit; no capability building	No money flow; contribution of skill/time/knowledge; empowerment check; dignity rules
NGO directories	List organizations	Static, no collaboration	Needs from communities themselves; live collaboration
Skill marketplaces	Hire talent, price competition	Commercial transaction	No bidding, no price; capability-to-need with handover
Event platforms	Ticketing, RSVPs	Attendance, not continuity or growth	Events feed circles, moments and journey
Social networks	Likes, followers, feeds	Ego, attention, comparison	Private by default, no public counts, small circles

USP: *The first platform where the interface itself refuses to make one human superior to another, and where every Seva is designed to leave both the person and the community more capable.*

---

33. Judge Q&A

Q: How is this not just a volunteering platform? A: There is no volunteer or beneficiary role in the data model. Needs are posted by the community, and every need requires an empowerment note. Reflection, circles and chains make it about growth and continuity, not tasks completed.

Q: How do you stop it becoming a gamified ego platform? A: No points, badges, public counts or leaderboards, and a CI check prevents us adding banned language. Journey is private.

Q: Isn't the AI Mirror a risk for sensitive reflections? A: It reads only the person's own text, never judges, never scores, is optional, validated by code, and the person can mark an insight "not quite". Reflections are encrypted at rest and admins cannot read them.

Q: How do you verify Seva actually happened? A: Creator completion plus participant confirmation, optional evidence, organization verification. Deliberately light, because surveillance contradicts non-attachment.

Q: How do you protect vulnerable communities? A: Community's own words, consent before contact, location rounding, dignity moderation checklist, no photos of identifiable people in need, restricted features for minors.

Q: Why would anyone come back without points? A: Relationships (circles), learning (the skills they want), continuity (next Seva), and seeing their own growth in the Journey. We measure return rate, not hours.

Q: What happens if nobody matches a need? A: It is flagged as a capability gap for admins and surfaced to nearby participants; the poster is never blamed or ranked.

Q: How would this scale to many cities or languages? A: See section 35. Location-aware discovery, i18n from day one, async AI translation, partitioned data.

Q: What is the business model? A: None assumed for the hackathon. Sustainable options consistent with the philosophy: grants, institutional partnerships with Mission centers and schools, optional organization subscriptions. Never ads or selling data.

Q: Where does Vivekananda's thought appear in the code? A: In the schema (no beneficiary field), the API (no ranking endpoint), the AI guardrails, the UI lint, and the metrics. Remove his name and the product still stands on dignity, equality and empowerment (Test 8).

---

34. Technical Risks and Mitigations

Risk	Mitigation
Cold start: no needs or participants	Seed realistic data; partner with one school or Mission center before launch; demo with real people
AI hallucination in need structuring	Always editable draft; poster confirms; validator on output
Mirror producing judgmental text	Strict prompt, output validator, retry then fallback, user feedback
Chat abuse	Reports, block, rate limits, circle admins, moderator review, no 1:1 for minors
Safety in real-world meetups	Public places default, consent, safety info required, organization verification, minors rules
Privacy breach	Column encryption, least privilege, audit log, no reflection logging
Over-scoping in a hackathon	Strict MVP loop, cut list, feature freeze, mock-first frontend
Frontend/backend drift	Shared contracts package, MSW mocks, OpenAPI diff in CI
LLM cost or outage	Caching, token limits, `AI_UNAVAILABLE` fallbacks
Real-time unreliability on free hosting	Poll fallback; persist messages first

---

35. Scalability

- **Start:** modular monolith, single Postgres, optional Redis. Enough for tens of thousands of users.
- **Search/discovery:** add PostGIS and a GIN index on skills; move to OpenSearch only if needed.
- **Real-time:** Socket.IO with the Redis adapter; sticky sessions or move chat to a managed service.

- **AI:** queue (BullMQ) for Mirror/summaries, caching, per-user rate limits, provider abstraction for cost and sovereignty.
- **Data:** partition messages by time; archive closed circles; read replicas for analytics views.
- **Multi-region and language:** i18n and RTL-ready, per-region moderation teams, region-scoped discovery defaults.
- **Institutions:** organization workspaces for schools and Mission centers, with role-limited admin tools (still no individual rankings).

---

36. Final Architecture Diagram (text form)

```
                 +-----------------------------+
                 |   Participants (web / PWA)   |
                 +--------------+--------------+
                                |
                   Next.js 14 (React, TS, Tailwind)
  Landing | Discover | Needs | Seva | Circles | Moment | Journey | Admin
        TanStack Query  |  MSW mocks (dev)  |  Socket.IO client
                                |
             HTTPS REST /api/v1   +   WebSocket /circles
                                |
+-------------------------------v-------------------------------+
|                  API (NestJS modular monolith)                |
|  auth | profiles | skills | sankalp | needs | seva | exchange |
|  matching | circles | messages | moments | journey          |
|  notifications | moderation | consents | admin                |
|  common: Zod validation, authz policies, error filter, logs   |
+--------+-----------------+-----------------+---------------+
         |                 |                 |
  PostgreSQL (Prisma)   Redis (sockets,   AIService (LLM provider)
  users, profiles,      rate limits)       - need structuring
  needs, seva, circles,                    - reflection summary
  moments (encrypted),                     - mirror + guardrail
  reports, audit                           (no training, no retention)
         |
  Object storage (optional evidence)

 Shared: /packages/contracts (Zod schemas, TS types, OpenAPI)
 CI/CD: GitHub Actions -> Vercel (web), Render/Railway (api), Neon (db)
```

```
  Product loop on top of the system:
  FEEL -> DISCOVER -> CONNECT -> SERVE -> LEARN -> REFLECT -> EMPOWER ->
CONTINUE
   |        |           |         |        |         |          |          |
  Sankalp  Needs +    Consent +  Real-    Moment:  Reflection  Empower-
Circle +
  + Need   explainable Circle   world    contrib. + Mirror    ment note
Chain +
  posting  matches               act      + learn  (private)   + handover
next Seva
```

---

37. Closing: Positioning Statement

What the judges should take away: this team did not build a volunteering platform and put Vivekananda's name on it. They translated his philosophy of Seva into data models (no beneficiary), interface language (no hero), AI behavior (mirror, not judge), metrics (return and continuity, not hours), and real-world action (every flow ends offline and returns as learning).

> **Technology is not the Seva. Technology is the enabler that helps human beings serve, learn, empower one another and continue the cycle of Seva.**